# Energy-Aware Task Scheduling for Edge AI Inference Using Lightweight Reinforcement Learning

Synthetic Research Manuscript • September 2026

**Research-status note:** This is an original synthetic example. Numerical results, datasets, experiments, and performance values are illustrative and are not empirical findings from a completed study.

**Abstract—** Edge artificial intelligence systems increasingly execute neural-network inference close to data sources to reduce latency and bandwidth consumption. However, heterogeneous edge devices have different compute capacities and energy costs, making static scheduling inefficient under changing workloads. This paper proposes a lightweight reinforcement learning scheduler that selects among local execution, neighboring-edge execution, and cloud offloading. The scheduler observes queue length, estimated inference latency, device utilization, and battery state, and optimizes a weighted objective containing latency, energy, and task-drop penalties. The evaluation uses synthetic trace-driven experiments to demonstrate the methodology and reporting structure.

**Keywords:** edge computing; AI inference; task scheduling; reinforcement learning; energy efficiency; offloading; latency

## 1. Introduction

Edge artificial intelligence places inference close to sensors and users in order to reduce network latency and bandwidth consumption. However, heterogeneous edge nodes have different compute capacities, energy costs, and queue conditions. A scheduler that optimizes only one metric can therefore perform poorly when workloads change.

This paper presents a synthetic study of a lightweight reinforcement learning scheduler that chooses between local execution, neighboring edge nodes, and cloud offloading. The agent observes queue length, compute utilization, network latency, battery state, and deadline pressure. The objective balances latency, energy, and task completion.

## 2. System Model

### 2.1 Edge Topology

The modeled system contains one gateway, three edge nodes, and a cloud service. Each arriving inference task has an estimated compute demand, input size, and deadline. The scheduler selects one of five destinations: local execution, Edge-1, Edge-2, Edge-3, or cloud.

### 2.2 State Representation

The state contains normalized gateway queue length, CPU utilization for each edge node, estimated network latency, battery state, and the fraction of tasks approaching deadlines. The resulting representation contains 18 dimensions.

### 2.3 Action Space

The action is discrete because exactly one execution destination is selected for each task. This makes the controller compatible with value-based reinforcement learning and relatively simple to deploy on a gateway.

## 3. Proposed Scheduling Method

### 3.1 Multi-Objective Reward

The illustrative reward is $r_t = -0.55L_t - 0.25E_t - 1.5D_t - 4F_t$, where $L_t$ is normalized latency, $E_t$ is energy consumption, $D_t$ is deadline pressure, and $F_t$ indicates task failure. Each term is bounded before aggregation.

### 3.2 Network Architecture

The scheduler uses a two-layer MLP with 128 and 64 hidden units. A dueling value head decomposes state value and action advantage. Experience replay stores 50,000 transitions and a target network is updated periodically.

### 3.3 Training Protocol

Training uses randomized workload traces containing low, medium, and high arrival rates. Network delay and node capacity are also randomized. Epsilon-greedy exploration decreases epsilon from 1.0 to 0.05. Evaluation uses fixed, unseen traces.

## 4. Experimental Design

The synthetic benchmark contains 100,000 inference tasks divided into 20 traces. Workloads include burst periods, idle periods, and changes in edge-node availability. Every method receives identical traces.

| Method | Decision rule | Learning |
|---|---|---|
| RL-Scheduler | Multi-objective Q policy | Yes |
| Round-Robin | Cyclic destination selection | No |
| Latency-Greedy | Minimum predicted latency | No |
| Energy-Greedy | Minimum predicted energy | No |

Table 1. Synthetic benchmark methods.

### 4.1 Metrics

The primary metrics are mean end-to-end latency, energy per completed task, task completion rate, and deadline violation rate. Robustness is evaluated under a workload distribution different from training.

## 5. Results

| Method | Latency (ms) | Energy (J/task) | Completion (%) | Violation (%) |
|---|---|---|---|---|
| RL-Scheduler | 84.6 | 2.71 | 97.8 | 4.1 |
| Round-Robin | 112.3 | 3.18 | 93.2 | 8.7 |
| Latency-Greedy | 76.4 | 3.92 | 96.9 | 5.6 |
| Energy-Greedy | 128.1 | 2.21 | 89.5 | 12.9 |

Table 2. Illustrative results on synthetic traces.

The example results illustrate a multi-objective trade-off. Latency-Greedy produces the lowest latency but higher energy consumption, while Energy-Greedy reduces energy at the cost of latency and completion. The RL scheduler represents an intermediate policy shaped by all four reward components.

### 5.1 Workload Shift

| Method | Training load completion (%) | Shifted load completion (%) |
|---|---|---|
| RL-Scheduler | 97.8 | 93.4 |
| Round-Robin | 93.2 | 89.1 |
| Latency-Greedy | 96.9 | 86.8 |
| Energy-Greedy | 89.5 | 82.6 |

Table 3. Illustrative robustness evaluation.

All methods degrade under distribution shift because queueing and capacity conditions differ from training. A real study would need many independently generated traces and statistical uncertainty before drawing conclusions about generalization.

## 6. Ablation Analysis

A reward ablation can test whether each objective changes behavior. Removing the energy term is expected to shift decisions toward powerful nodes, while removing deadline pressure can increase late tasks during bursts.

| Reward variant | Latency (ms) | Energy (J/task) | Violation (%) |
|---|---|---|---|
| Full reward | 84.6 | 2.71 | 4.1 |
| No energy term | 79.2 | 3.61 | 4.5 |
| No deadline term | 82.8 | 2.64 | 9.8 |
| Latency only | 75.9 | 3.89 | 7.1 |

Table 4. Illustrative ablation results.

## 7. Discussion

The principal challenge is objective balancing. A scalar weighted reward converts multiple goals into one optimization target, but the weights determine the resulting trade-off. They should therefore be justified by application requirements rather than tuned solely for benchmark performance.

Deployment introduces additional effects such as thermal throttling, memory contention, accelerator sharing, and correlated network loss. A robust evaluation should include measured power, real network traces, multiple hardware configurations, and confidence intervals.

## 8. Limitations and Future Work

The workload traces and numerical results in this manuscript are synthetic. Future research should use hardware-in-the-loop measurements and investigate constrained reinforcement learning, uncertainty-aware scheduling, and hierarchical resource allocation across CPU, GPU, and NPU resources.

## 9. Conclusion

This paper described a lightweight reinforcement learning approach for energy-aware edge AI scheduling. By representing latency, energy, deadline pressure, and task failure in a bounded reward, the scheduler can be evaluated as a multi-objective decision system. The synthetic results demonstrate an appropriate reporting structure without presenting illustrative numbers as measured evidence.

## References

[1] M. Satyanarayanan, “The emergence of edge computing,” Computer, vol. 50, no. 1, pp. 30–39, 2017.

[2] W. Shi et al., “Edge computing: Vision and challenges,” IEEE Internet of Things Journal, vol. 3, no. 5, pp. 637–646, 2016.

[3] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, “A survey on mobile edge computing,” IEEE Communications Surveys & Tutorials, vol. 19, no. 4, pp. 2322–2358, 2017.

[4] V. Mnih et al., “Human-level control through deep reinforcement learning,” Nature, vol. 518, pp. 529–533, 2015.

[5] Z. Wang et al., “Dueling network architectures for deep reinforcement learning,” in Proc. ICML, 2016.

[6] T. Schaul et al., “Prioritized experience replay,” arXiv:1511.05952, 2015.

[7] J. Schulman et al., “Proximal policy optimization algorithms,” arXiv:1707.06347, 2017.

[8] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. MIT Press, 2018.